even with a smooth, bowl shaped function like the one in figure 16.10.

Figure 16.11 is an attempt of capturing how random sampling (which doesn’t perform any optimization after initialization) works.

A randomized heuristic is like tossing dozens, hundreds, or thousands of marbles at our muddy track (without being able to see where the finish line is): they stay exactly where they land, and in the end, we just choose the one that landed the closest to the finish line. The key is doing many attempts, hoping that at least one will land close enough to the optimal solution. Of course, the track could be so long that no matter how many times we try, we won’t be able to get any closer to a good solution: that’s the case for exponential problems, where the number of possible solutions is huge, when the input is large enough.

Moreover, we need to be careful using a completely randomized algorithm: we might toss several marbles in the same position: we could generate the same solution more than once.

If there is a good thing about this approach (besides being extremely cheap) is that the shape of the cost function doesn’t matter: we don’t need to engineer a good “track”, because there isn’t going to be any “marble rolling” after initialization.

At the same time, this is also the worst part: we can’t take advantage if we have a good cost function that smoothly degrades towards a global minimum.

If we think about marble races, we can perhaps find a workaround for that: if you ever played with marble on the beach, you probably know what to do when one is stuck in a pit of the sand track: you give the marble a nudge to get it out of there and start rolling again.